



Model Training and Results for the Recommender System

A technical review of the explicit-feedback MF and implicit-feedback ALS models trained on MovieLens ml-32m, covering methodology, experimental design, quantitative results, and production use

Prepared by bir.mov Project

Report date July 17, 2026

Models reviewed ml32m-mf128-v1 and ml32m-als128-v1

The experimental values in this report were verified against saved notebook outputs and model metadata. The training pipeline was not rerun while preparing this document.

Contents

Executive Summary	ii
1 Project Objective and Model Architecture	1
2 Dataset and Preprocessing	1
2.1 MovieLens ml-32m overview	1
2.2 Rare-film filtering and indexing	2
2.3 Per-user chronological split	2
3 Evaluation Protocol	2
3.1 Rating-prediction metrics	2
3.2 Top-N ranking metrics	3
4 Model 1: Explicit-Feedback Matrix Factorization	3
4.1 Model and objective	3
4.2 Training configuration	4
4.3 Training behavior	4
5 Model 2: Implicit-Feedback ALS	4
5.1 Converting explicit ratings into implicit preferences	4
5.2 Training configuration and artifact	5
6 Experimental Results and Comparison	5
6.1 Rating prediction	5
6.2 Top-10 ranking	6
6.3 Task-specific model selection	7
7 New-User Adaptation and Production Flow	7
7.1 Closed-form MF fold-in	7
7.2 ALS user recalculation	7
7.3 Hybrid reranking	7
8 Limitations and Threats to Validity	8
9 Conclusion and Recommended Next Steps	9
A Reproducibility and Source Traceability	9
A.1 Recorded training environment	10
A.2 Source-file mapping	10

Executive Summary

bir.mov uses a hybrid recommendation architecture that assigns two distinct tasks to two specialized models. **ALS-128** ranks unseen candidate films from the user's positive history. **MF-128** predicts the 0.5-5.0 star rating that the user is likely to assign to each candidate. The offline results directly support this separation of responsibilities.

Key finding	Result	Interpretation
MF rating prediction	Test RMSE: 0.8244	5.9% lower than the bias baseline
ALS Top-10 ranking	Recall@10: 0.0842	72.2% above the popularity baseline
ALS Top-10 ranking	NDCG@10: 0.0668	55.3% above the popularity baseline
MF Top-10 ranking	Recall@10: 0.0005	Accurate ratings, but unsuitable for direct catalog ranking

The central result is the alignment between optimization objective and production role. Explicit-feedback MF minimizes star-rating error, while implicit-feedback ALS is optimized to retrieve liked films near the top of the list. The production implementation follows this evidence: ALS generates a broad candidate pool, MF assigns personalized rating estimates, and a final layer combines genre preference, IMDb quality, similarity to liked and disliked films, novelty, and list diversity.

One limitation is essential to the interpretation of this report. The offline experiment evaluates the base MF and ALS models. The notebook does not contain a separate end-to-end test of the production hybrid reranker. The report therefore does not claim a measured numerical gain for that final layer.

1 Project Objective and Model Architecture

bir.mov generates personalized movie recommendations by matching ratings exported from IMDb or Letterboxd to the MovieLens catalog. The system answers two different questions with two different models:

1. **Which films should rank highest?** Implicit-feedback ALS produces a candidate ranking from the user's positive history.
2. **What rating will the user give this film?** Explicit-feedback MF predicts a personal score on the star-rating scale.

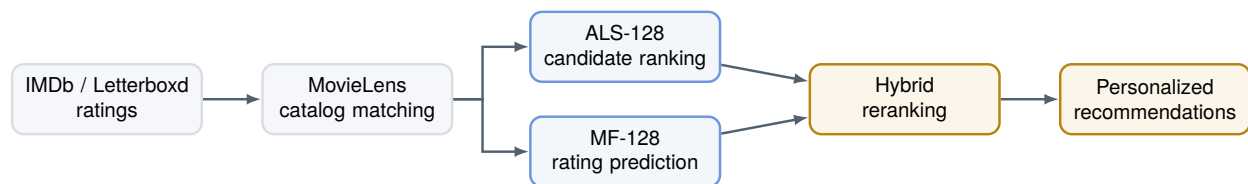


Figure 1: Separation of responsibilities between the two production models.

This architecture is designed for new-user inference. A person uploading ratings is not represented in the original MovieLens user vocabulary. At request time, the system recomputes the person's representation against fixed movie factors. Section 7 describes this *fold-in* process in detail.

2 Dataset and Preprocessing

2.1 MovieLens ml-32m overview

The training source is MovieLens ml-32m [1]. Table 1 summarizes the raw tables recorded in the training notebook.

Table 1: Key properties of the raw dataset.

Property	Value
Users	200,948
Rated films	84,432
Catalog size in <code>movies.csv</code>	87,585
Ratings	32,000,204
Tag applications	2,000,072
Mean ratings per user	159.2
Rating mean / median / standard deviation	3.540 / 3.5 / 1.059
Rating time span	1995-2023
User-item matrix sparsity	99.8114%

Source: Recorded outputs in Section 1 of `train/ml-32m-train.ipynb`.

The data exhibits a pronounced long tail. The median film has only 5 ratings, while the most popular film has 102,929. The top 1% of films account for 54.8% of all ratings. This structure makes poorly supported movie factors unstable and helps explain the extreme predictions observed when MF is used as a direct full-catalog ranker.

2.2 Rare-film filtering and indexing

Films with fewer than 5 ratings were removed from the model vocabulary. The filter discards 48.0% of rated films while retaining 99.75% of all rating events. The resulting model matrix contains 200,948 users and 43,884 films.

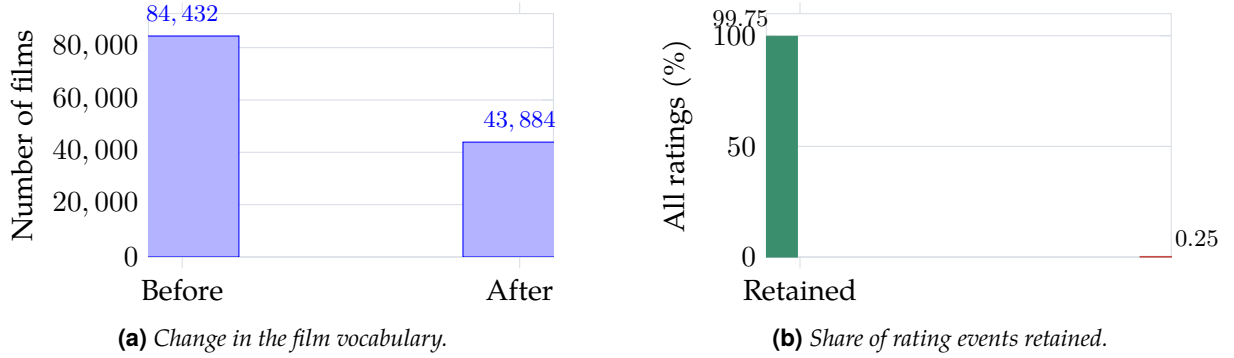


Figure 2: Effect of the minimum-5-ratings filter.

MovieLens identifiers were converted to contiguous zero-based indices required by sparse matrices and embedding layers. The `user_ids.npy` and `movie_ids.npy` files preserve the mapping back to the original MovieLens identifiers.

2.3 Per-user chronological split

Ratings were sorted by timestamp within each user. The first 80% were assigned to training, the next 10% to validation, and the remainder to testing. A per-user chronological split uses the past to predict the future and reduces the future-information leakage associated with random splitting.

Table 2: Chronological split after filtering.

Split	Ratings	Share
Training	25,614,481	80.24%
Validation	3,200,365	10.03%
Test	3,106,621	9.73%

The training CSR matrix has a density of only 0.2905%. This level of sparsity motivates low-dimensional latent-factor models and the sparse linear-algebra advantages of ALS.

3 Evaluation Protocol

The models optimize different objectives, so two metric families were used.

3.1 Rating-prediction metrics

MF and the rating baselines were evaluated from the error between the observed rating r_{ui} and the estimate $\hat{r}_{ui}$:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{(u,i)} (r_{ui} - \hat{r}_{ui})^2}, \quad (1)$$

$$\text{MAE} = \frac{1}{N} \sum_{(u,i)} |r_{ui} - \hat{r}_{ui}|. \quad (2)$$

RMSE places greater weight on large errors, whereas MAE gives a direct average absolute deviation on the star scale. Lower values are better for both metrics. Predictions were clipped to 0.5-5.0 during evaluation.

3.2 Top-N ranking metrics

A test item was considered relevant when its rating was at least 4.0. Items observed in the training set were excluded from the candidate list. Of the 187,278 users with at least one relevant test item, 10,000 were sampled with a fixed seed. The experiment used $K = 10$:

- **Recall@10** measures the share of a user’s relevant test films recovered in the first 10 recommendations.
- **NDCG@10** rewards relevant films not only for appearing in the list, but also for appearing near the top.

MF, ALS, and the popularity baseline were evaluated with the same users, relevance definition, and K . This shared protocol is a major strength of the comparison.

Interpretation note. RMSE and Recall@10 do not measure the same capability. A model can predict star ratings accurately on average while failing to select the best 10 films from tens of thousands of candidates. The MF results demonstrate precisely this distinction.

4 Model 1: Explicit-Feedback Matrix Factorization

4.1 Model and objective

The MF model learns a 128-dimensional latent vector for every user and film. Its prediction equation is [2]:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i, \quad (3)$$

where μ is the training-set mean, b_u and b_i are user and item biases, and $\mathbf{p}_u$ and $\mathbf{q}_i$ are user and movie factors. The observed training mean was $\mu = 3.5587$.

The biases were not initialized randomly. A regularized bias baseline was first fitted with $\lambda_u = \lambda_i = 10$, and its user and item biases were copied into the MF layers as a warm start. Latent factors were initialized from a normal distribution with standard deviation 0.05.

The implemented mini-batch loss adds an L2 penalty on the active user and movie factors to the mean squared error:

$$\mathcal{L}_{\text{MF}} = \frac{1}{|B|} \sum_{(u,i) \in B} (r_{ui} - \hat{r}_{ui})^2 + \lambda_f \frac{1}{|B|} \sum_{(u,i) \in B} (\|\mathbf{p}_u\|_2^2 + \|\mathbf{q}_i\|_2^2). \quad (4)$$

4.2 Training configuration

Table 3: MF-128 training configuration.

Setting	Value
Latent factors	128
Optimizer	Adam
Factor learning rate	0.005
Bias learning rate	0.0005
Factor L2 coefficient λ_f	0.02
Mini-batch size	16,384
Maximum epochs	20
Early-stopping patience	3 epochs
Random seed	0
Compute device	Apple MPS

Factors use the higher learning rate, while warm-started biases use a learning rate ten times smaller. The early-stopping counter advances when validation RMSE fails to improve by more than 10^{-4} and stops training after three consecutive non-improving epochs.

4.3 Training behavior

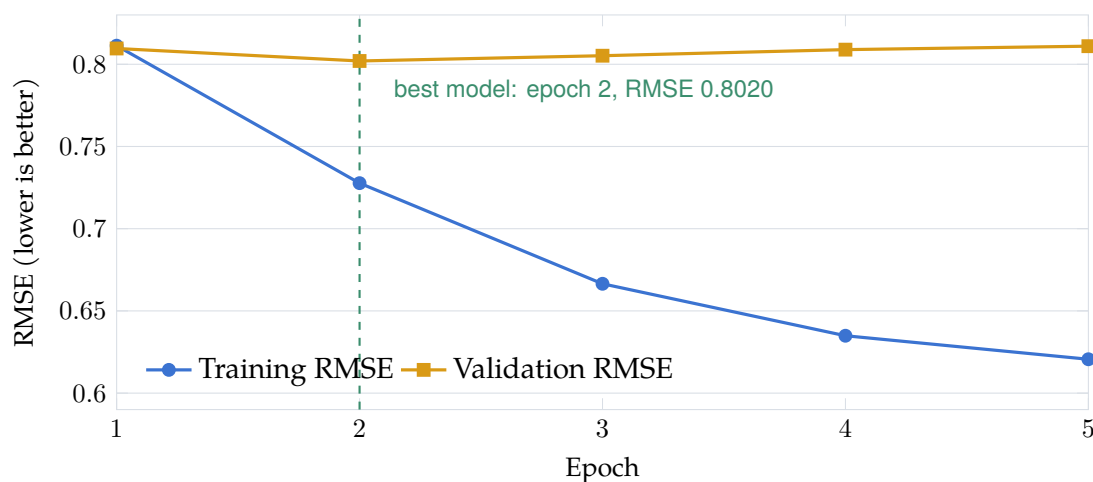


Figure 3: MF training and validation curves.

Source: Epochs 1-5 recorded in the training notebook.

Training error decreases continuously, while validation error begins to rise after epoch 2. This divergence is an early sign of overfitting. The epoch-2 weights were retained, and training stopped after epoch 5.

5 Model 2: Implicit-Feedback ALS

5.1 Converting explicit ratings into implicit preferences

For ALS, training ratings of 4.0 or higher were treated as positive interactions. This produced 12,979,522 positive examples, representing 50.7% of all training ratings. A total of 200,588 users had at least one positive interaction.

Each positive cell received a confidence weight of $\alpha = 40$. The model was trained with the weighted least-squares formulation for implicit feedback [3]:

$$\min_{\mathbf{x}, \mathbf{y}} \sum_{u,i} c_{ui} \left(p_{ui} - \mathbf{x}_u^\top \mathbf{y}_i \right)^2 + \lambda \left(\sum_u \|\mathbf{x}_u\|_2^2 + \sum_i \|\mathbf{y}_i\|_2^2 \right), \quad (5)$$

where p_{ui} represents the observed preference, c_{ui} its confidence, and $\mathbf{x}_u$ and $\mathbf{y}_i$ the user and item factors. Alternating Least Squares alternates between regularized closed-form updates for user factors with movie factors fixed and movie factors with user factors fixed.

5.2 Training configuration and artifact

Table 4: ALS-128 training configuration.

Setting	Value
Latent factors	128
Iterations	20
Regularization	0.05
Positive-interaction threshold	$r_{ui} \geq 4.0$
Confidence weight α	40.0
Positive training interactions	12,979,522
Random seed	0
Threads	All available cores
Recorded training time	3 minutes 40 seconds

The deployed artifact stores a $43,884 \times 128$ movie-factor matrix. Full MovieLens user factors are not required for new-user fold-in, so they were replaced by a 1×128 placeholder before saving. This keeps the ALS artifact at approximately 21.4 MiB, compared with approximately 120.5 MiB for the MF artifact containing all trained user and movie embeddings.

6 Experimental Results and Comparison

6.1 Rating prediction

Table 5: Rating-prediction results. Lower is better.

Model	Validation		Test	
	RMSE	MAE	RMSE	MAE
Global mean	1.0573	0.8272	1.0571	0.8267
User + item bias	0.8633	0.6537	0.8763	0.6631
MF-128	0.8020	0.5998	0.8244	0.6159

Source: Sections 3.1, 3.2, and 4.4 of `train/ml-32m-train.ipynb`.

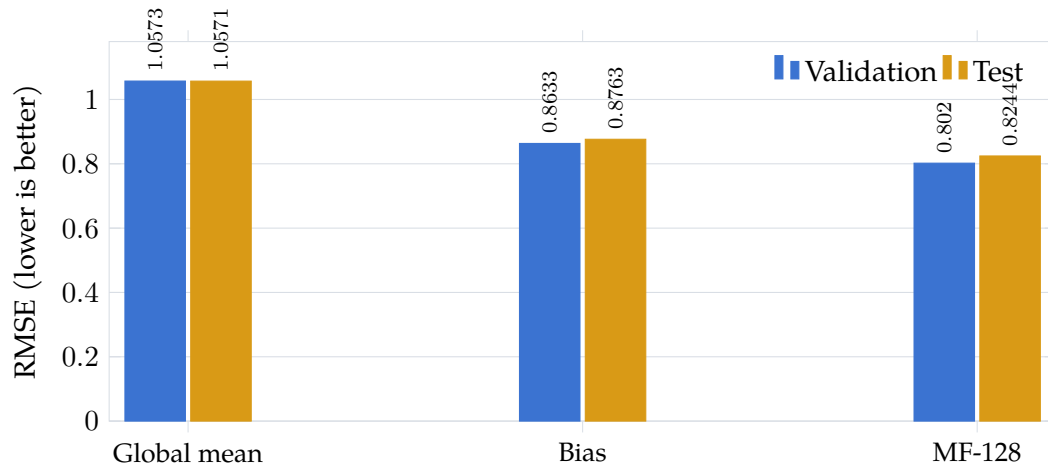


Figure 4: RMSE comparison for rating prediction.

MF-128 reduces test RMSE by 5.9% and test MAE by 7.1% relative to the bias baseline. The gain over the global mean is larger. Test RMSE is 0.0224 higher than validation RMSE, which is consistent with a more difficult chronological test period.

6.2 Top-10 ranking

Table 6: Top-10 ranking results for 10,000 users. Higher is better.

Method	Recall@10	NDCG@10	Evaluated users
Popularity baseline	0.0489	0.0430	10,000
Direct MF-128 ranking	0.0005	0.0005	10,000
ALS-128	0.0842	0.0668	10,000

Source: Table values are rounded to four decimals; relative changes use the full-precision notebook outputs.

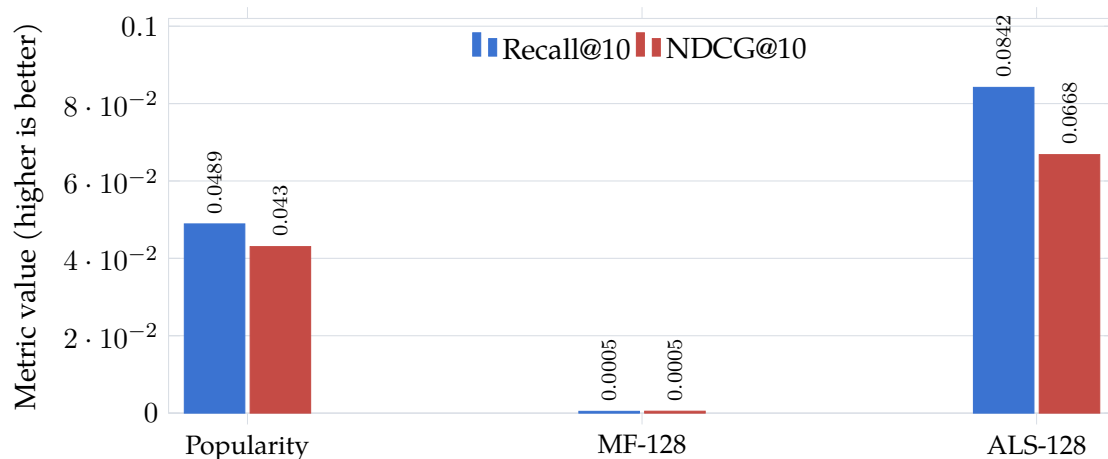


Figure 5: Top-10 ranking quality.

ALS improves Recall@10 by 72.2% and NDCG@10 by 55.3% relative to the popularity baseline. The gains show that personalized retrieval is substantially better than recommending the same popular films to every user.

Direct MF ranking is poor: its Recall@10 is about one 185th of the ALS result. The notebook's qualitative diagnosis shows that the top MF results frequently contain films with only a few or

a few dozen training ratings. Large dot products from weakly supported movie factors disrupt full-catalog ranking despite good average rating-prediction accuracy. This result supports using MF for score prediction and ALS for candidate generation.

6.3 Task-specific model selection

Table 7: *Experimental rationale for the model assignment.*

Task	Selected model	Rationale
Candidate rank- ing	ALS-128	Best Recall@10 and NDCG@10; strong personalization gain over popularity
Rating predic- tion	MF-128	Best validation and test RMSE/MAE; direct estimates on the 0.5-5.0 scale
Full-catalog MF ranking	Not used	Extreme scores on rare films and very low Top-10 metrics

7 New-User Adaptation and Production Flow

The trained user factors do not include a person who uploads an IMDb or Letterboxd file after deployment. The runtime solves a new representation against the fixed movie factors without retraining either model.

7.1 Closed-form MF fold-in

For n matched ratings, the system first estimates a damped user bias:

$$b_u = \frac{\sum_{i \in I_u} (r_{ui} - \mu - b_i)}{n + 10}. \quad (6)$$

It then uses the observed movie-factor matrix Q_u and residual target $\mathbf{y}$ to solve the new user factor:

$$\mathbf{p}_u = \left(Q_u^\top Q_u + \lambda_u I \right)^{-1} Q_u^\top \mathbf{y}, \quad \lambda_u = \max(1, 0.02n). \quad (7)$$

The factor produces predictions for all films, clipped to 0.5-5.0. The interface rounds the Letterboxd view to half-star increments and converts the same prediction to a 10-point IMDb-style display.

7.2 ALS user recalculation

If the training-time 4.0 like threshold does not produce enough positive examples for a new user, the runtime relaxes the threshold to 3.5 and then 3.0, seeking at least five liked films. The liked films form a one-row sparse matrix with confidence 40. The `implicit library's recalculate_user=True` option solves a user factor while keeping the trained movie factors fixed. All films already rated by the user are excluded from recommendations.

7.3 Hybrid reranking

ALS generates a pool larger than the requested recommendation list. Under the standard configuration, the pool is at least 500 films, at most 4,000, and approximately 40 times the requested list length. The final score combines the components in Table 8.

Table 8: *Single-user reranking components.*

Component	Weight
ALS candidate score	+0.50
MF personalized rating prediction	+0.22
Genre preference	+0.10
IMDb quality and vote confidence	+0.08
Similarity to liked films	+0.08
Novelty	+0.02
Similarity to disliked films	−0.16

A diversity penalty similar to maximal marginal relevance reduces near-duplicate recommendations in the final list. The “because you liked” explanations use cosine similarity between normalized ALS movie factors.

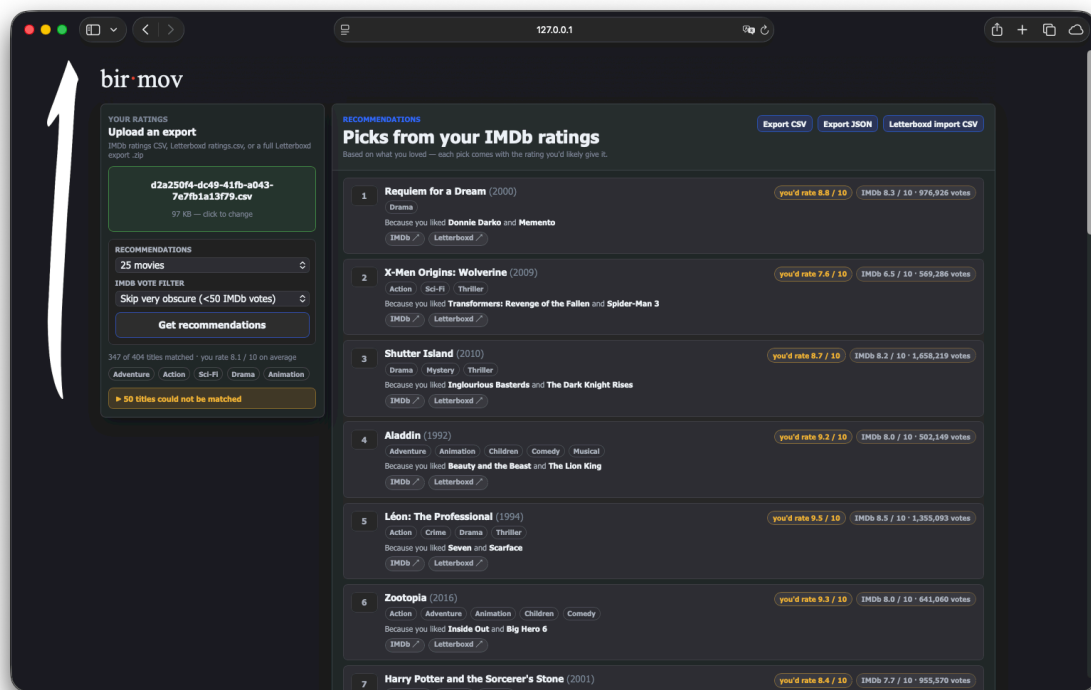


Figure 6: *The bir.mov interface using both models. The gold badge reflects the MF rating prediction, while the recommendation order reflects the hybrid ranking.*

Source: Project file docs/screenshot.png.

8 Limitations and Threats to Validity

- **Single random seed:** MF and ALS use seed 0, and ranking evaluation also samples users with seed 0. Multiple runs and confidence intervals were not reported, so sensitivity to randomness remains unmeasured.
- **Hyperparameter search:** The notebook does not record a systematic grid search, Bayesian optimization, or cross-validation study. The selected settings are a valid operating point, but are not proven globally optimal.
- **Information loss in ALS:** Ratings of 4.0 and 5.0 become the same positive signal. Rating mag-

nitude and low-rating negative evidence are not used by the base ALS training objective.

- **Ranking candidate set:** Test evaluation excludes training items, but validation items are not separately removed. Alternative candidate definitions can change the absolute metric levels.
- **Long-tail tradeoff:** The minimum-5-ratings filter reduces noise but removes new and highly niche films from the vocabulary. Production IMDb vote filters also influence the balance between discovery and confidence.
- **Hybrid-layer evaluation:** The reranking weights are defined in production code, but the combined list has not been separately measured for Recall/NDCG, diversity, novelty, and user satisfaction in this experiment.
- **Temporal and community bias:** MovieLens represents interactions from 1995-2023 and its own user community. Generalization to current releases, countries, languages, and demographic groups requires separate validation.
- **No online measurement:** Offline gains do not guarantee higher click, watch, or satisfaction rates. No A/B test or shadow-traffic evaluation has been recorded.

These limitations do not invalidate the task assignment between the models; they identify the most important directions for further evaluation.

9 Conclusion and Recommended Next Steps

The experiments reveal two clearly different strengths. MF-128 is the best rating predictor, with test RMSE 0.8244 and MAE 0.6159. ALS-128 is clearly superior for Top-10 retrieval, achieving Recall@10 of 0.0842 and NDCG@10 of 0.0668, respectively 72.2% and 55.3% above the popularity baseline. MF's failure as a full-catalog ranker demonstrates that low RMSE alone does not imply a useful recommendation list.

The current assignment is therefore technically justified: **ALS finds candidates; MF predicts ratings.** Runtime fold-in adapts both models to a new user without full retraining, while the hybrid layer adds quality, genre, similarity, negative-preference, and diversity signals.

The recommended sequence of follow-up experiments is:

1. Compare the hybrid reranker with pure ALS under the same chronological protocol using Recall@10, NDCG@10, catalog coverage, novelty, and intra-list diversity.
2. Learn reranking weights on validation data instead of fixing them manually, and evaluate group-recommendation weights separately.
3. Run multi-seed searches over ALS confidence, regularization, factor count, and like threshold, as well as MF factor count and L2 strength.
4. Investigate the rare-film score explosions in direct MF ranking with minimum-support constraints, factor-norm clipping, and ranking objectives such as BPR.
5. Evaluate fold-in quality by user-history length, particularly for 1-4, 5-19, and 20+ matched ratings.
6. Validate offline gains against real user behavior through a privacy-preserving online experiment.

A Reproducibility and Source Traceability

A.1 Recorded training environment

Table 9: *Training environment recorded in the notebook.*

Component	Version / value
Python	3.14.6
Platform	macOS 26.5.1, arm64
NumPy	2.5.0
pandas	3.0.3
SciPy	1.18.0
Matplotlib	3.11.0
PyTorch	2.12.1
implicit	0.7.3
CUDA / MPS	No / Yes
Selected device	MPS
PyTorch threads	4

A.2 Source-file mapping

File	Role in this report
<code>train/ml-32m-train.ipynb</code>	Data analysis, filtering, chronological split, baselines, MF and ALS training, and recorded metrics
<code>model/ml32m-mf128-v1/meta.json</code>	MF artifact name, dimensions, factor count, and exact best validation RMSE
<code>model/ml32m-als128-v1/meta.json</code>	ALS iterations, regularization, confidence, positive-example count, and artifact scope
<code>app/recommender.py</code>	New-user fold-in equations, candidate generation, and hybrid reranking weights
<code>README.md</code>	Project purpose, model roles, and deployment overview

The model artifacts carry a saved-at timestamp of July 2, 2026 UTC. This report was prepared on July 17, 2026 by reading the files above. Because the raw MovieLens files and training pipeline were not rerun, the values presented here are a traceable technical summary of the recorded experiment rather than a new execution.

References

- [1] F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context”, *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, 2015. [doi:10.1145/2827872](https://doi.org/10.1145/2827872).
- [2] Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems”, *Computer*, vol. 42, no. 8, pp. 30-37, 2009. [doi:10.1109/MC.2009.263](https://doi.org/10.1109/MC.2009.263).
- [3] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative Filtering for Implicit Feedback Datasets”, *2008 Eighth IEEE International Conference on Data Mining*, pp. 263-272, 2008. [doi:10.1109/ICDM.2008.22](https://doi.org/10.1109/ICDM.2008.22).
- [4] GroupLens Research, “MovieLens 32M Dataset”. <https://grouplens.org/datasets/movielens/32m/>.
- [5] bir.mov Project, “MovieLens ml-32m Recommendation Model Pipeline”, `train/ml-32m-train.ipynb`, recorded notebook in the project workspace.